

ESCUELA TECNOLÓGICA INSTITUTO TÉCNICO CENTRAL

Tecnología en Desarrollo de Software
Electiva Técnica II - Ciencia de Datos

Optimización de rutas de última milla

Caso LogiExpress

Aplicación sobre el dataset Olist (Kaggle)



Integrante 1: Jhon Alejandro Correal Martinez

Integrante 2: Rafael Andres Guzman Rodriguez

Docente: Elias Buitrago

Semestre: Séptimo

Fecha: 9 de junio de 2026

Repositorio GitHub: <https://github.com/ACJUGGLER645/oli>

Resumen

Este informe documenta el desarrollo del caso LogiExpress: una empresa ficticia que debe entregar 30 pedidos en la ciudad de Sao Paulo desde un unico depposito, respetando una capacidad maxima por vehiculo. Se trabajo con el dataset publico de Olist (Kaggle), se calcularon las distancias entre puntos usando la formula de Haversine, se demostro experimentalmente que la solucion exacta no es viable para 30 pedidos y se construyeron las rutas con dos heurísticas sencillas: vecino mas cercano y 2-opt. El resultado final fueron **4 rutas factibles** con una distancia total de **161.60 km**, una mejora de 6.92 km frente al ordenamiento inicial.

Índice

1. Planteamiento del problema	2
2. El dataset: de donde salen los datos	2
3. Preparacion de los datos	3
4. Medicion de distancia entre puntos geograficos	3
5. Por que no podemos probar todas las rutas posibles	4
6. La regla del peso por vehiculo	5
7. Construccion de las rutas	6
7.1. Paso 1. Barrido angular	6
7.2. Paso 2. Vecino mas cercano (NN)	6
7.3. Paso 3. Refinamiento local con 2-opt	6
8. Resultados	7
9. Notebooks empleados	8
10. Limitaciones	9
11. Conclusiones	9

1. Planteamiento del problema

LogiExpress es una operacion logistica ficticia que debe atender un conjunto de pedidos desde un unico centro de distribucion (deposito) al que cada vehiculo regresa una vez completada su ruta. Cada pedido tiene una ubicacion geografica y un peso conocidos. Cada vehiculo dispone de una capacidad maxima de carga.

El problema requiere satisfacer simultaneamente dos condiciones:

- **Factibilidad de carga:** la suma de los pesos de los pedidos asignados a un vehiculo no debe superar la capacidad del mismo.
- **Minimizacion de la distancia total recorrida,** asociada a una reduccion proporcional del consumo de combustible, tiempo operativo y emisiones.

Este escenario corresponde a una version simplificada del *Problema de Ruteo de Vehiculos con Capacidad*, conocido por sus siglas en ingles como CVRP (*Capacitated Vehicle Routing Problem*), una generalizacion del clasico Problema del Vendedor Viajero (TSP) que incorpora la restriccion de capacidad.

2. El dataset: de donde salen los datos

Nombre: Brazilian E-Commerce Public Dataset by Olist.

Fuente: Kaggle

Enlace: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Olist es una marketplace de Brasil (algo parecido a Mercado Libre o Amazon). Publicaron en Kaggle un archivo abierto con casi 100 000 pedidos reales hechos entre 2016 y 2018. Los datos estan anonimizados: no aparecen nombres ni direcciones exactas, solo prefijos postales.

El dataset viene en **cinco archivos** que hay que combinar para poder armar un solo pedido con todo lo que necesitamos:

- `olist_orders_dataset.csv` - id del pedido y su estado.
- `olist_customers_dataset.csv` - cliente y prefijo postal.
- `olist_order_items_dataset.csv` - productos del pedido.
- `olist_products_dataset.csv` - peso de cada producto.
- `olist_geolocation_dataset.csv` - latitud/longitud por prefijo postal.

Tabla	Filas	Columnas
0	ordenes	99441
1	clientes	99441
2	items	112850
3	productos	32951
4	geolocalizacion	1000163

order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
0	e481151cdd54678b7cc49136f2d5a17	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	2017-10-18 00:00:00
1	53c0b2c58bc7dce0b674e2150273451	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	2018-08-13 00:00:00
2	47770e9100c2d3c44946d9d07a69d	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:50:00	2018-08-17 18:06:29	2018-09-04 00:00:00

customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0	0688999e2b1a1fbc88172c00a8bc7	861e84711a542a4b53843c5d57f6bb0	franca	SP
1	18955e63d337b6b2d0e7b18a428ac77	290c77bc52967ac935093aa66c333dc3	sao bernardo do campo	SP
2	4e7b3a002858e8ebd8712fda0374a03	060a732b5b29a8181a18229c7b0b2b5a	sao paulo	SP

order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value
0	00010242f6b5a6d1ba2d792cb16214	1	4244733a05a7ecb4970a9e2663c13a61	49436dada16ac8b2bce089ec2a041202	2017-09-19 09:45:35	58.9
1	00018772f0320c507190d7a14bddd3	1	e5f2d52b8d2189ee58885ca93d83a8f	d67d6d04e1b6c2b814352b383a6d2d36	2017-05-03 11:05:13	239.9
2	000229c398224e8ca0657d44e703a	1	c777355d18672b67abbea9d944d9d3	5d51032e9d3242ad084c38acab86d23d	2018-01-18 14:48:30	199.0

product_id	product_category_name	product_name_length	product_description_length	product_photos_qty	product_weight_g	product_length_cm	product_height_cm	product_width_cm
0	1e9e8a04dc84541ed28657ea517e5	perfumaria	40.0	287.0	1.0	225.0	16.0	14.0
1	3aa071139cb18a67ca9e5dea641aa2f	artes	44.0	276.0	1.0	1000.0	30.0	20.0
2	98bd78ac0810374ed1b65e2919757171	esporte_lazer	46.0	250.0	1.0	154.0	18.0	9.0

geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
0	1037	-23.545621	-46.639292	sao paulo
1	1046	-23.546081	-46.644820	sao paulo
2	1046	-23.546129	-46.642951	sao paulo

Figura 1: Dataset cargado en el notebook.

3. Preparacion de los datos

Los archivos originales no se pueden usar directamente. Hay que hacer varios pasos de limpieza:

1. **Unir las tablas:** se cruzan **orders** con **customers**, luego con **items**, productos y geolocalizacion. Asi cada fila queda con *id del pedido*, *coordenadas* y *peso* en un solo lugar.
2. **Resumir la geolocalizacion:** cada prefijo postal tiene muchas coordenadas distintas en el dataset (porque cubre varias cuadras). Tomamos la *mediana* de latitud y longitud por prefijo para representar el centro de esa zona.
3. **Calcular el peso del pedido:** sumamos el peso (en gramos) de los productos que lo componen y lo pasamos a kilogramos.
4. **Filtrar:** solo pedidos con estado **delivered** (entregados), ciudad Sao Paulo, peso mayor que cero, coordenadas dentro de Brasil.
5. **Seleccionar 30 pedidos:** se tomo una muestra aleatoria de pedidos con peso entre el percentil 50 y 90 (es decir, ni los mas livianos ni los mas pesados).

Que es un percentil

Si ordenas todos los pedidos del mas liviano al mas pesado, el percentil 50 es el peso del pedido que queda en la mitad. El percentil 90 es el que esta despues del 90 % de los pedidos. Tomar el rango 50–90 quiere decir: descarto los pedidos demasiado livianos (que no llenarian el vehiculo) y los demasiado pesados (que copan un vehiculo con uno o dos pedidos).

```
... Using Colab cache for faster access to the 'brazilian-ecommerce' dataset.
Descarga con kagglehub revisada en: /kaggle/input/brazilian-ecommerce
Usando dataset demo: False
Pedidos cargados: 30
Depósito definido: [-23.5675533902995, -46.64456321120669]
```

	Pedido_ID	Latitud	Longitud	Peso_kg	Ciudad	Estado	Cantidad_items
0	4888d985060eb4760da7373673343df2	-23.537368	-46.659642	1.60	sao paulo	SP	1
1	42e9b8d9802096afe88a27ff32fb838a	-23.665189	-46.646753	2.55	sao paulo	SP	1
2	a2a04ead650a00874aec73c0d989298b	-23.520018	-46.719062	1.00	sao paulo	SP	1
3	0c6715b2f888887f2ab9c7d6e099d38b	-23.523154	-46.445408	0.70	sao paulo	SP	1
4	4f6b1944e5dab6270cb4010b00edb3bc	-23.569763	-46.656772	0.70	sao paulo	SP	1
5	da8831dfbb89ea6b128840224899b348	-23.564658	-46.596266	0.98	sao paulo	SP	1
6	4ac657f0949f8f58a10b48c192df0b4e	-23.594000	-46.751503	1.01	sao paulo	SP	1
7	1882714a6a534ed3eae539fe6f17a1b1	-23.606953	-46.523468	2.08	sao paulo	SP	1
8	6d0d1b394c3e9a042ef3fba7cb0968f5	-23.624000	-46.675273	1.98	sao paulo	SP	1
9	d61c0e8066e7a5a2e6071533b9efdd95	-23.701208	-46.634834	1.08	sao paulo	SP	2
10	0d21189c5494d8288ee358b3b24f85b4	-23.585995	-46.734721	1.15	sao paulo	SP	1

Figura 2: Tabla final con los 30 pedidos seleccionados y sus columnas: id, latitud, longitud, peso, ciudad, estado.

4. Medicion de distancia entre puntos geograficos

La construccion de rutas requiere conocer la distancia entre cada par de puntos. Una aproximacion inicial consistiria en aplicar el teorema de Pitagoras sobre las coordenadas (latitud y longitud) tratandolas como un plano cartesiano. Este procedimiento es incorrecto porque la superficie terrestre no es plana: la longitud de un grado de longitud varia con la latitud (es maxima en el ecuador y nula en los polos), por lo que la aproximacion plana acumula error sistematico.

Por lo tanto se emplea la **formula de Haversine**, que calcula la distancia geodesica entre dos puntos sobre la superficie de una esfera (modelo esferico de la Tierra):

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\varphi_2 - \varphi_1}{2} \right) + \cos \varphi_1 \cos \varphi_2 \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right) \quad (1)$$

Desglose de componentes

- d : la distancia entre los dos puntos, en kilometros. Es lo que queremos calcular.
- R : el radio de la Tierra. Usamos $R \approx 6371$ km.
- φ_1 y φ_2 : la *latitud* de cada punto (que tan al norte o al sur estan), expresada en radianes.
- λ_1 y λ_2 : la *longitud* de cada punto (que tan al este o al oeste estan), en radianes.
- $\sin$, $\cos$, $\arcsin$: funciones de trigonometria de secundaria.

La formula resuelve un triangulo sobre la esfera y devuelve la distancia mas corta que se podria recorrer sobre la superficie.

Con esta formula calculamos la distancia entre cada par de puntos: 30 pedidos mas el deposito dan 31 puntos, asi que la *matriz de distancias* es una tabla de 31×31 casillas. Se calcula una sola vez al principio y los algoritmos posteriores solo la consultan.

Limitacion. Haversine da la distancia en *linea recta* sobre la curvatura terrestre. No tiene en cuenta calles, sentidos viales, semaforos ni trafico. En la realidad la distancia recorrida sera mayor.

id																														
Nodo		0	1	2	3	4	5	6	7	8	9	...	21	22	23	24	25	26	27	28	29	30								
0	0.000000	3.691594	10.858911	9.252620	20.893043	1.268373	4.932985	11.288008	13.094823	7.013388	...	2.070734	11.336726	4.600225	18.059056	13.766974	4.922043	0.177931	7.374937	6.306208	11.425326									
1	3.691594	0.000000	14.273578	6.357644	21.898055	3.613950	7.137235	11.283290	15.889549	9.763807	...	3.015255	11.440765	8.059771	21.069003	11.069003	2.030629	3.520324	4.312828	8.600224	9.773940									
2	10.858911	14.273578	0.000000	17.744436	25.891783	10.659926	12.305222	13.286618	14.129927	5.423660	...	11.384241	19.554525	6.261634	7.861923	24.498985	14.634979	11.033945	18.233239	12.516121	21.574867									
3	9.252620	6.357644	17.744436	0.000000	27.902734	8.421010	13.465927	8.866073	22.155622	12.393624	...	7.412243	17.191906	12.326682	23.019015	13.416801	4.450593	9.129790	7.905852	14.933547	14.106380									
4	20.893043	21.898055	25.891783	27.902734	0.000000	22.160316	16.055854	32.178561	12.252487	25.972271	...	22.796624	10.786739	22.623758	33.376837	19.212815	23.921374	20.880241	20.423848	14.900956	15.379008									

5 rows x 31 columns

Figura 3: Primeras filas de la matriz de distancias Haversine de tamaño 31×31 generada por el notebook.

5. Por que no podemos probar todas las rutas posibles

La forma “perfecta” de resolver el problema seria *probar todas las rutas posibles* y quedarse con la mas corta. Pero la cantidad de combinaciones crece monstruosamente rapido con el numero de pedidos.

Si tenemos n pedidos, la cantidad de ordenes distintos en que podemos visitarlos es $n!$ (se lee “n factorial”).

Que es n factorial

$n!$ significa multiplicar $1 \times 2 \times 3 \times \dots \times n$. Por ejemplo:

- $3! = 1 \times 2 \times 3 = 6$
- $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$
- $8! = 40320$
- $30! \approx 2,65 \times 10^{32}$ (un 265 seguido de 30 ceros)

Aun si un computador pudiera revisar mil millones de rutas por segundo, para 30 pedidos terminaria *mucho despues* de que el sol se apague. Por eso esta tecnica solo sirve para problemas muy pequenos.

En el notebook se hizo el experimento con n entre 3 y 8 pedidos:

Conclusion: para 30 pedidos no podemos buscar la ruta perfecta. Hay que conformarse con una ruta *suficientemente buena* usando **heurísticas**.

Cuadro 1: Tiempo medido al probar todas las rutas posibles (fuerza bruta).

n	Rutas a probar	Tiempo (s)	Distancia (km)
3	6	0,0003	58,13
4	24	0,0012	65,01
5	120	0,0063	65,80
6	720	0,0502	68,00
7	5 040	0,3036	68,07
8	40 320	2,7162	68,08

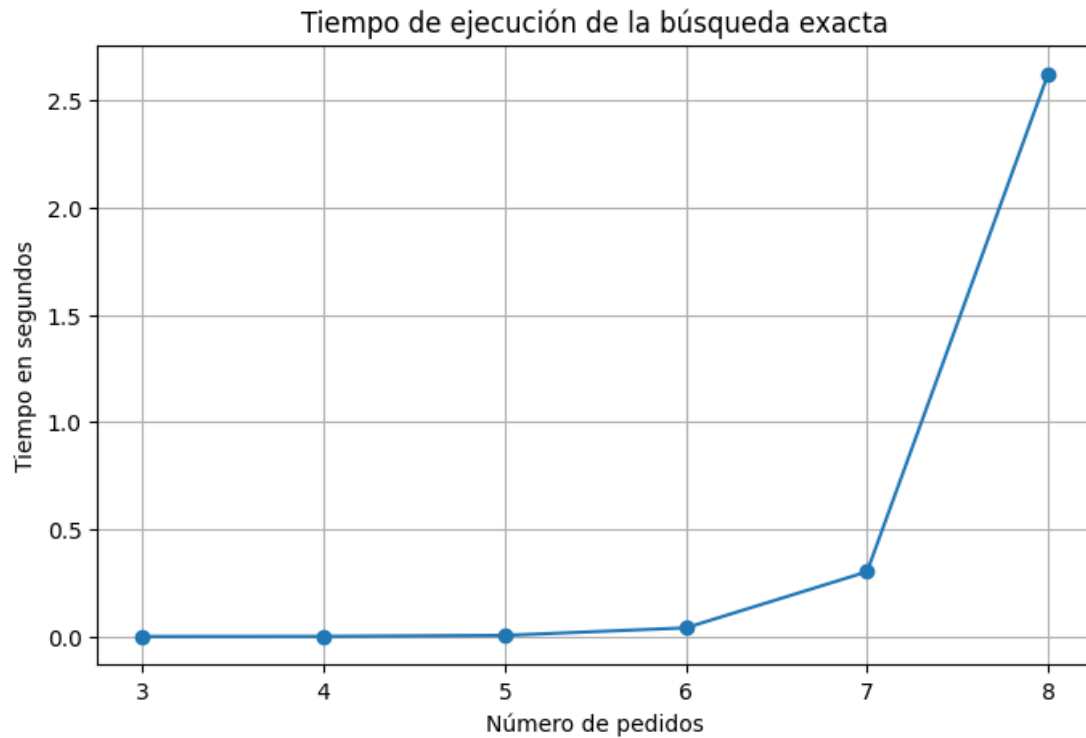


Figura 4: Grafica del tiempo de ejecución contra el numero de pedidos. La curva crece muy rapido.

Definición de heurística

Procedimiento algorítmico que produce soluciones aproximadas a un problema de optimización en tiempo polinomial, sin garantizar la obtención del óptimo global. Se utiliza cuando la búsqueda exacta resulta computacionalmente inviable. En este informe se aplican dos heurísticas en cascada: *vecino mas cercano* (NN) para una solución inicial y *2-opt* para refinamiento local.

6. La regla del peso por vehiculo

Cada vehiculo solo puede cargar hasta C kilogramos. Para cada ruta r formada por los pedidos que ese vehiculo visita, la suma de los pesos no puede pasarse del limite:

$$\sum_{i \in r} \text{Peso}_i \leq C \quad (2)$$

Desglose de componentes

- r : una ruta, es decir, el grupo de pedidos que asignamos a un vehiculo.
- $i \in r$: cada pedido i que pertenece a esa ruta.
- Peso_i : el peso (en kilogramos) del pedido i .
- $\sum_{i \in r}$: el simbolo $\sum$ (sigma) significa *sumar todo*. Aqui sumamos los pesos de todos los pedidos de la ruta.
- C : la capacidad maxima del vehiculo, en kilogramos.

La formula entonces se lee: “la suma de los pesos de los pedidos de la ruta tiene que ser menor o igual a la capacidad del vehiculo”.

Decision tomada. La guia sugiere $C = 100$ kg. Pero los productos de Olist son pequenos (libros, ropa, accesorios) y con esa capacidad los 30 pedidos cabian en un solo vehiculo, por lo que no podiamos mostrar como funciona la restriccion. Cambiamos a $C = 15$ kg, que es coherente con una flota de motos o bicicletas electricas para reparto urbano de productos pequenos.

7. Construccion de las rutas

7.1. Paso 1. Barrido angular

El *sweep heuristic* calcula, para cada pedido, el angulo polar respecto al deposito. Los pedidos se ordenan por ese angulo (por ejemplo, en sentido horario) y se asignan secuencialmente a rutas: se incorpora un pedido a la ruta actual mientras el peso acumulado no supere la capacidad; al excederla se cierra la ruta y se inicia otra. El procedimiento tiende a agrupar pedidos geograficamente cercanos en el mismo vehiculo.

7.2. Paso 2. Vecino mas cercano (NN)

Definidos los grupos, NN determina el orden de visita dentro de cada ruta mediante una estrategia voraz: desde el nodo actual se selecciona el pedido no visitado mas cercano segun la matriz de distancias, y se repite el proceso hasta agotar el grupo, regresando al deposito. Su complejidad es $\mathcal{O}(n^2)$. Es eficiente pero subóptima: frecuentemente produce tramos que se cruzan entre si.

7.3. Paso 3. Refinamiento local con 2-opt

El algoritmo 2-opt examina pares de aristas de la ruta y evalua si la inversion del segmento intermedio reduce la distancia total. Si la reduccion es positiva, aplica el cambio; en caso contrario, prueba el siguiente par. El proceso se repite hasta que ningun intercambio reduzca la distancia: en ese punto se ha alcanzado un optimo local.

Origen del nombre 2-opt

En cada movimiento se modifican exactamente *dos aristas* de la ruta: se eliminan dos conexiones existentes y se sustituyen por una configuracion alternativa que invierte el segmento entre ellas. Geometricamente, el procedimiento elimina cruces de aristas en el plano: una interseccion entre dos tramos puede deshacerse invirtiendo el segmento intermedio, lo que reduce la distancia total cuando se cumple la desigualdad triangular.

Solucion optima frente a solucion heuristica. La solucion *optima global* es la de menor distancia entre todas las configuraciones posibles. Una *heuristica* entrega una solucion de calidad aceptable sin garantia de optimalidad. 2-opt converge a un *optimo local*: una ruta cuya distancia

no puede reducirse mediante ningun intercambio de dos aristas, aunque puedan existir mejores soluciones inalcanzables por este tipo de movimiento.

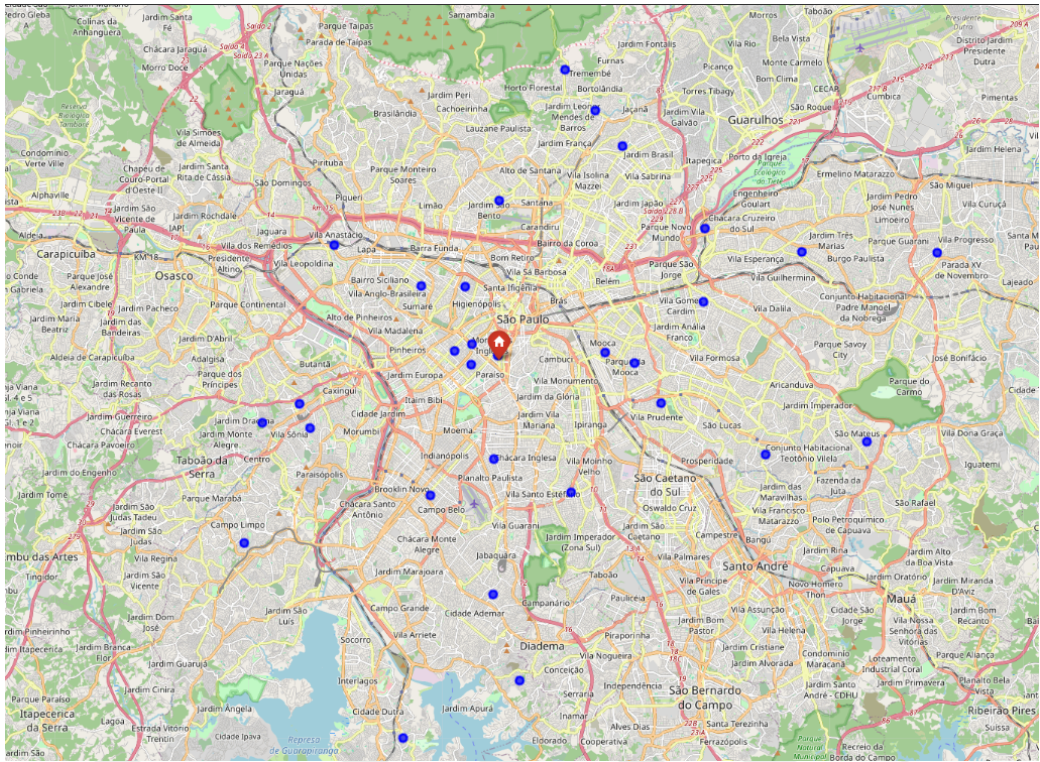


Figura 5: Mapa de los 30 pedidos seleccionados, con el deposito marcado en rojo.

8. Resultados

Cuadro 2: Resumen de las rutas generadas (capacidad = 15 kg).

Vehiculo	Pedidos	Peso (kg)	NN (km)	2-opt (km)	Mejora (km)
1	11	14,48	64,55	59,58	4,98
2	9	13,01	55,14	55,14	0,00
3	8	14,64	44,47	42,53	1,94
4	2	3,44	4,35	4,35	0,00
Total	30	45,57	168,51	161,60	6,92

Lectura de los resultados.

- Se generaron 4 vehiculos. Tres de ellos cargan cerca del limite (14 kg de los 15 kg posibles); un cuarto vehiculo lleva los 2 pedidos sobrantes.
- 2-opt logro mejorar las dos rutas con mas pedidos (vehiculos 1 y 3). Las rutas cortas (vehiculos 2 y 4) ya estaban en su minimo local y no admitieron mejora.
- La mejora total fue de 6,92 km, equivalente a un **4,1 %** menos respecto al ordenamiento inicial. Es una mejora modesta pero ganada en milisegundos de calculo.

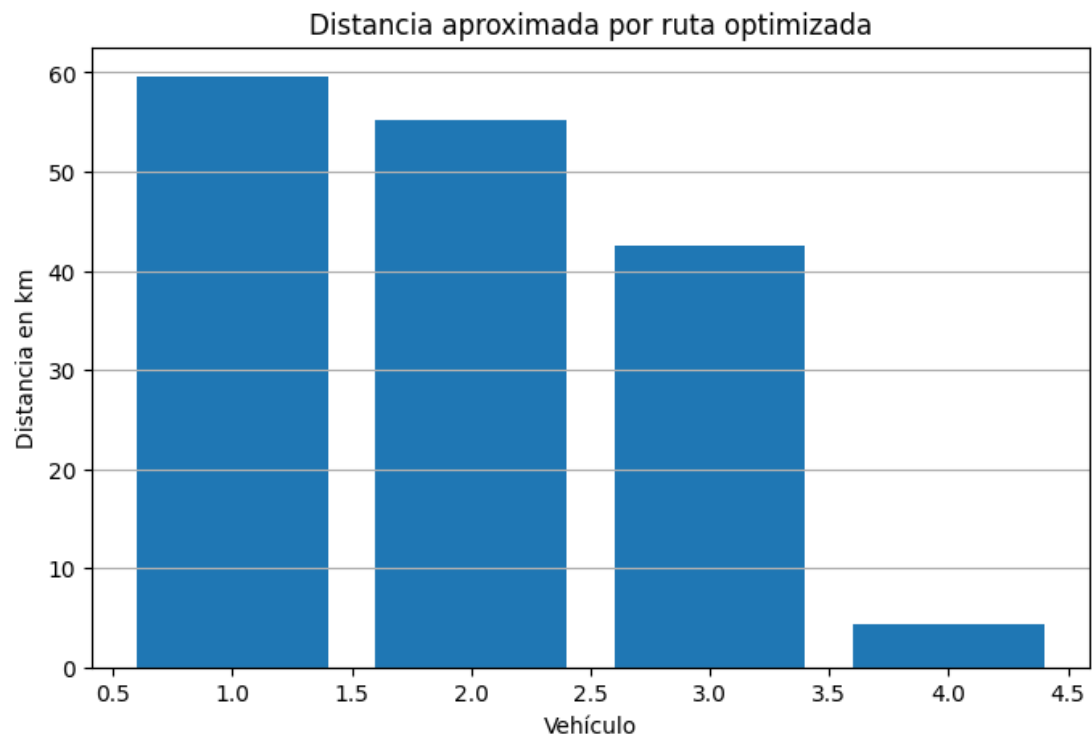


Figura 6: Distancia por vehiculo despues de aplicar 2-opt.

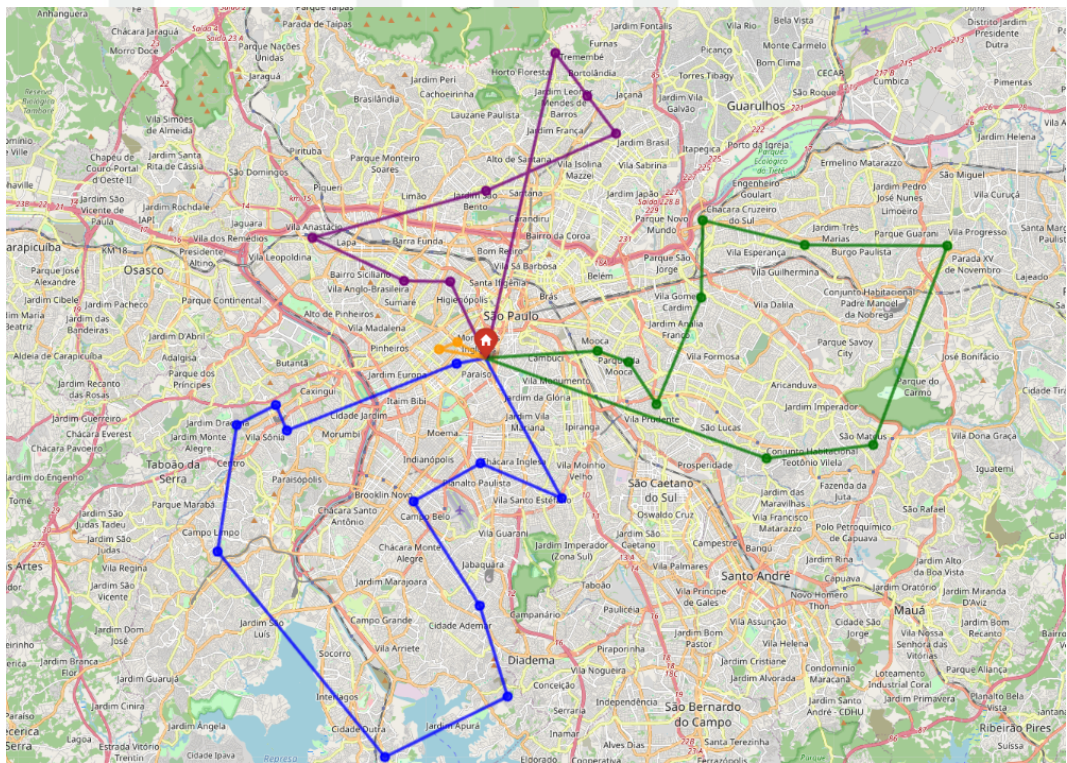


Figura 7: Mapa final con las cuatro rutas, una de cada color, y el deposito en rojo.

9. Notebooks empleados

El notebook de partida es el propuesto por el docente del curso (*Optimizacion de rutas de ultima milla*, autor: Elias Buitrago Bolivar). El equipo decidio **conservar el notebook original**

sin ejecutar como evidencia de la version recibida y entregar las modificaciones en un segundo notebook ejecutado. Asimismo, los datos empleados son los indicados por el docente: el dataset publico de Olist de Kaggle, con foco en la ciudad de Sao Paulo.

Cuadro 3: Cambios introducidos en la version propuesta respecto al notebook original.

Aspecto	Notebook original	Notebook propuesto (v3)
Identificacion	Sin seccion de autores	Celda inicial con programa, integrantes y docente
Configuracion Colab	Instalaciones <i>inline</i> genericas	Seccion dedicada con carga de <code>kaggle.json</code> via <code>google.colab.files</code>
Capacidad por vehiculo	$C = 100$ kg	$C = 15$ kg (justificado para reparto urbano de productos livianos)
Seleccion de la muestra	Filtro general por ciudad y estado de entrega	Adicionalmente: peso en percentil 50–90 y muestra aleatoria de 30 pedidos con <code>random_state=42</code>
Interpretacion de complejidad	Plantilla en blanco	Texto completo redactado por el equipo
Seccion de informe final	No incluida	Bloque anadido al final con la lectura de resultados
Estado de ejecucion	Sin ejecutar	Ejecutado de principio a fin (4 rutas, 161,60 km)

El resto de la logica (Haversine, matriz de distancias, fuerza bruta acotada, barrido angular, NN y 2-opt) se conserva del notebook original; las modificaciones se limitan a parametrizacion, justificacion del rango de pesos y redaccion del informe embebido.

10. Limitaciones

- Haversine entrega distancia en linea recta sobre la esfera. La distancia real por calles es mayor (semaforos, sentidos, rodeos).
- No se modelaron horarios de entrega, prioridades, tiempos de descarga ni jornada del conductor.
- Asumimos que todos los vehiculos son iguales. En la realidad suele haber flota mixta.
- El deposito es ficticio: se ubico en el centro geometrico de los pedidos.
- La geolocalizacion es por prefijo postal, no por direccion exacta del cliente.
- 2-opt encuentra un buen local, no garantiza el mejor global.
- Los datos son de 2016–2018; el comportamiento real evoluciona.

11. Conclusiones

- Logramos llevar un dataset publico real de principio a fin: lo descargamos, limpiamos, integramos, calculamos distancias y armamos rutas factibles.
- Confirmamos con un experimento que probar todas las rutas posibles para 30 pedidos es imposible en la practica. Por eso las heurísticas son necesarias en logistica real.
- La combinacion *barrido angular + agrupar por capacidad + vecino mas cercano + 2-opt* entrega rutas razonables en segundos y respeta la restriccion de carga.
- Decisiones como bajar la capacidad a 15 kg y filtrar el rango 50–90 del peso fueron necesarias para que el ejercicio muestre la dinamica del problema real.

- Para acercarlo a una operacion real habria que sumar: ventanas horarias, distancias por calles, flota mixta y prioridades de cliente.

Anexo. Estructura del repositorio

- `notebooks/original.ipynb` - notebook original del docente, conservado sin ejecutar.
- `notebooks/Optimizacion_Rutasv3.ipynb` - notebook propuesto, ejecutado con salidas embebidas.
- `data/pedidos_logiexpress_limpio.csv` - 30 pedidos seleccionados.
- `data/matriz_distancias_haversine.csv` - matriz 31×31 de distancias geodesicas.
- `informe.tex` / `informe.pdf` - codigo fuente y PDF del informe.
- `index.html` - version web del informe (GitHub Pages).
- `README.md` - documentacion principal del repositorio.
- `assets/` - imagenes incluidas en el informe (marca de agua y capturas).

ETITC